Mini-Project 1: Introduction to MATLAB Mobile

Goals

This mini-project will introduce you to MATLAB mobile, a tool which allows you to collect data using sensors on your smartphone and stream it to applications for data analysis implemented in MATLAB. In this mini-project you will set-up MATLAB mobile on your smartphone and learn how to stream data from the accelerometer on your phone to a MATLAB application running on your personal computer or the cloud.

Step 1: Installation

Step 1 will guide you through installing MATLAB on your personal computer and MATLAB mobile on your smartphone.

Step 1.1: Create Mathworks Account

MATLAB is freely available for all UC Davis undergraduates for educational purposes. *You do not need to purchase a MATLAB license for this class.* To activate your mathworks account, go to software.ucdavis.edu, search for “MATLAB” and follow the instructions provided in “MATLAB for Undergraduate Students” as shown in Figure 1.

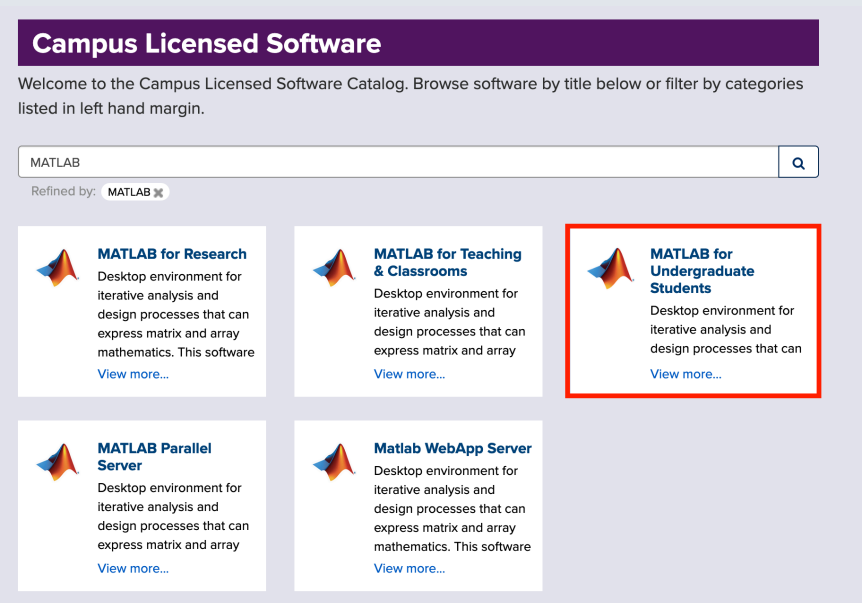

Figure 1: Creating a Mathworks account and installing MATLAB.

MATLAB can be used either by installing it directly onto your PC (instructions are provided in the link above) or via the cloud at https://matlab.mathworks.com/. You are encouraged to use the

cloud version for this class. The cloud version is platform agnostic and makes it easier to share data with your teammates in the final project.

Checkpoint: navigate to https://matlab.mathworks.com/ and launch an instance of MATLAB. You should see a MATLAB GUI similar to Figure 2. Enter a simple command like “3*4” in the command window and inspect the output.

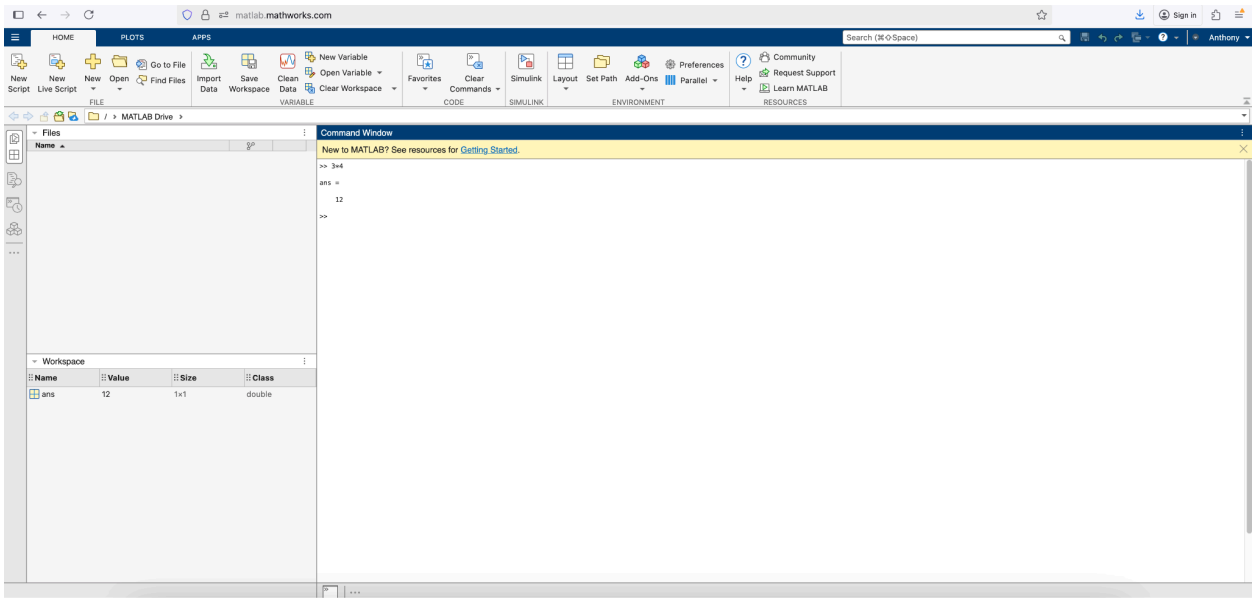

Figure 2: MATLAB online command window.

Step 1.2: Install MATLAB Mobile

Install Matlab mobile on your smartphone by searching for it in your app-store (links are provided at the bottom of https://www.mathworks.com/products/matlab-mobile.html) and following your standard installation process. Once installation is complete, open the app and sign in using your UC Davis email. You should see a simplified command window similar to what is shown in Figure 3 below.

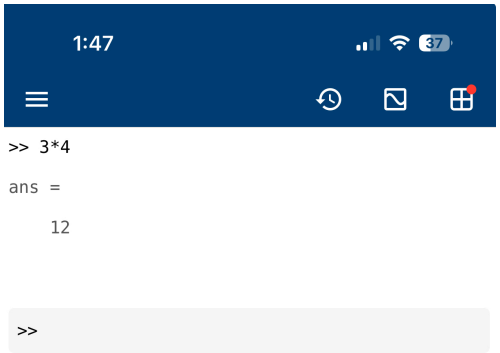

Figure 3: MATLAB Mobile Command Window

Step 2: Create Logging and Analysis Script

Step 2 will guide you through the process of writing a simple script to stream data from the accelerometer on your smartphone to an analysis application on MATLAB. Pay careful attention to this process. Over the remainder of the quarter, we will gradually increase its complexity by adding more sensors and more sophisticated MATLAB applications.

Step 2.1: First Logging Session

In this step, we will run a simple, interactive logging session from the command line. First we need to grant MATLAB permission to access your phone's sensors. From the menu, click on "Sensors > More" and toggle "Sensor Access" under "MATLAB Remote Access as shown in Figure 4 below.

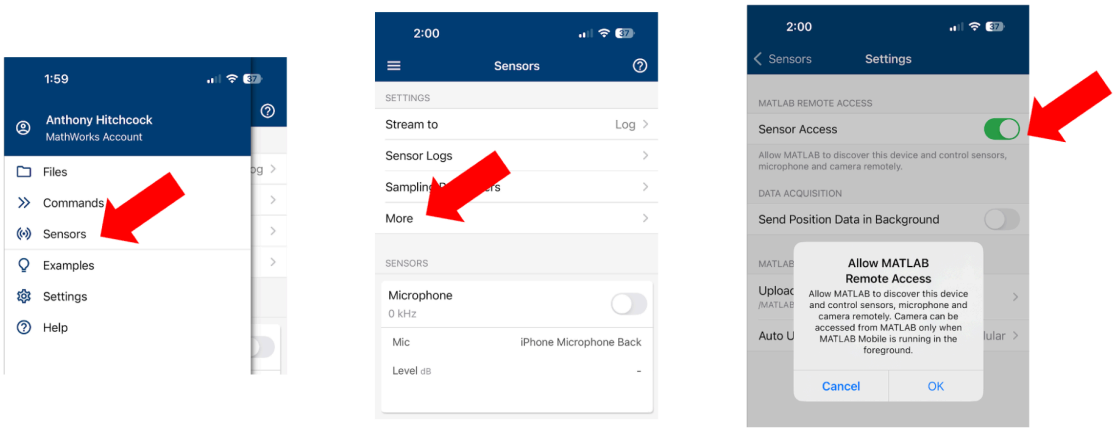

Figure 4: Configuring sensor permissions

Under "Sensors" also toggle the "Acceleration" sensor to enable collection. In the MATLAB command window on your browser (i.e. on your laptop/PC) type the following commands:

```
m = mobiledev
m.Logging = 1
```

Wave your phone around in front of you for a few seconds and then type:

```
m.Logging = 0
```

The first command creates a connection to MATLAB mobile, the second starts streaming sensor data over this connection, and the third stops streaming. To inspect the raw sensor values, type

```
accellog(m)
```

You should see a bunch of numbers (technically a 2D array) like what is shown in Figure 5 below.

```
Command Window
New to MATLAB? See resources for Getting Started.
>> m.Logging = 0

m =

mobiledev with properties:

                      Device: 'iPhone - iPhone'
                   Connected: 1
                     Logging: 0
             AvailableCameras: {'back' 'front'}
         AvailableMicrophones: {'AirPods Pro ' 'iPhone Microphone Back' 'iPhone Microphone Bottom' 'iPhone Microphone Front'}
           SelectedMicrophone: 'iPhone Microphone Back'

             InitialTimestamp: '08-Jul-2025 14:14:22.034'

   AccelerationSensorEnabled: 1    (135 Logged values)
AngularVelocitySensorEnabled: 0
       MagneticSensorEnabled: 0
    OrientationSensorEnabled: 0
       PositionSensorEnabled: 0
          MicrophoneEnabled: 0
Show all properties
>> accellog(m)

ans =

   -2.4859    3.9497    7.1324
   -2.6399    4.9245    8.2962
   -2.0654    5.7242    7.9003
   -2.1307    5.4580    7.8359
   -2.1394    5.3454    7.7956
   -2.3456    5.0984    8.3038
   -2.0132    4.8170    8.5059
   -1.9705    4.9120    8.3830
   -2.0060    5.2173    7.8441
   -2.1051    5.4020    7.5458
   -2.2621    5.4442    7.8567
   -2.0007    5.5009    8.0031
   -2.4911    5.4255    7.8558
   -2.4062    5.5588    7.5620
   -2.0644    5.6018    7.7971
   -1.8708    5.5802    7.9036
   -2.2378    5.6516    7.7808
   -2.0082    5.7946    8.3152
   -1.9624    6.2318    8.1669
   -0.9023    6.0428    8.0166
   -0.7077    6.9357    6.7023
    0.9779    6.9289    7.0553
    1.0531    6.7302    7.9842
    0.0241    6.5752    7.9100
   -0.7456    6.7242    6.3425
```

Figure 5: Example of an acceleration sensor log. Each column indicates the measured acceleration in the X,Y, and Z directions respectively.

Step 2.2: Write a Logging Script

Typing commands out in the command line is tedious and error-prone. A much better way is to write a *script*. A script is simply a text file that contains a sequence of MATLAB commands (i.e. a program). Create a script called by clicking on the “New Script” button in MATLAB.

Write a script that performs the following actions:

1. Creates a connection to MATLAB mobile and assigns it to a variable “m”
2. Starts sensor logging
3. Collects data for exactly 10 seconds
4. Stops sensor logging

Hints:

- To pause MATLAB for 10 seconds, use the pause function.

- To run your script, you need to save it. As your projects become more complicated it will become more important to keep your code well organized. A good way to do this is to create a separate folder for each project. Create a folder “ENG6” in MATLAB drive. Within ENG6, create a new folder “MiniProject1.” Save your script as “logging.m” in the folder you just created. **You must name your script logging.m .**
- To run your script, you also need to set Matlab’s working directory to the folder “MiniProject1.” To do so, click on the caret as shown in Figure 6 and set your working directory to the folder in which you saved your script.

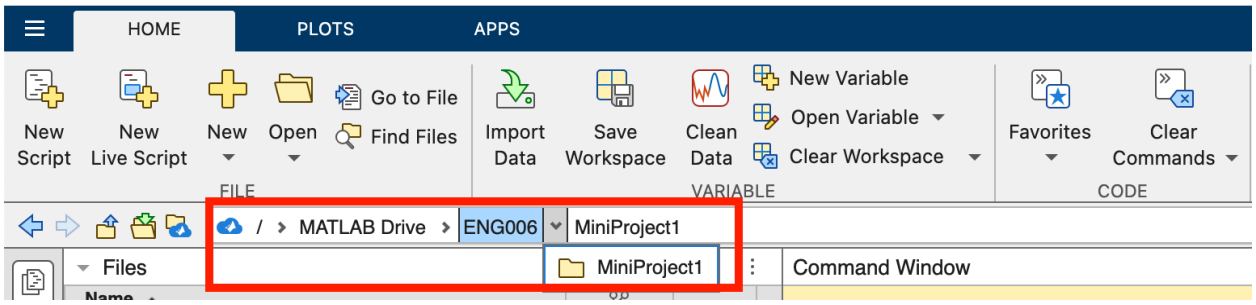

Figure 6: Setting the MATLAB working directory.

Step 3: Visualizing and Interpreting Sensor Readings

The raw sensor data collected in Step (2) is difficult for humans to interpret. An important component of any data analysis project is to create graphics and other summaries of data that are easily understood by humans. These can be used to look for patterns that might be useful/informative or as a sanity check on the measured values.

Download the script “plotAccelerometer.m” from Canvas and place it in the same folder as your script “logging.m” created in Step (2). The script will plot the triaxial accelerometer readings collected during a MATLAB mobile logging session and save them in a png file “accelerometerReadings.png” in your current working directory. Call the script from “logging.m” by typing:

```
plotAccelerometer
```

at the end of your script.

Step 3.1: Determining X, Y, and Z Axes on Your Phone

The triaxial accelerometer measures X, Y, and Z acceleration. But what does your phone consider to be the X, Y, and Z axes? To answer this question, hold your phone directly in front of you (with the screen facing you) and run a logging session while moving your phone along each axis: up/down, left/right, in/out. Plot the measured accelerometer values and use these to determine the X, Y, and Z axes on your phone. Try to accelerate along only one axis at a time,

and use swift, large, motions that will be easy to see in the plot. An example is shown in Figure 7 below. Your figure will look different.

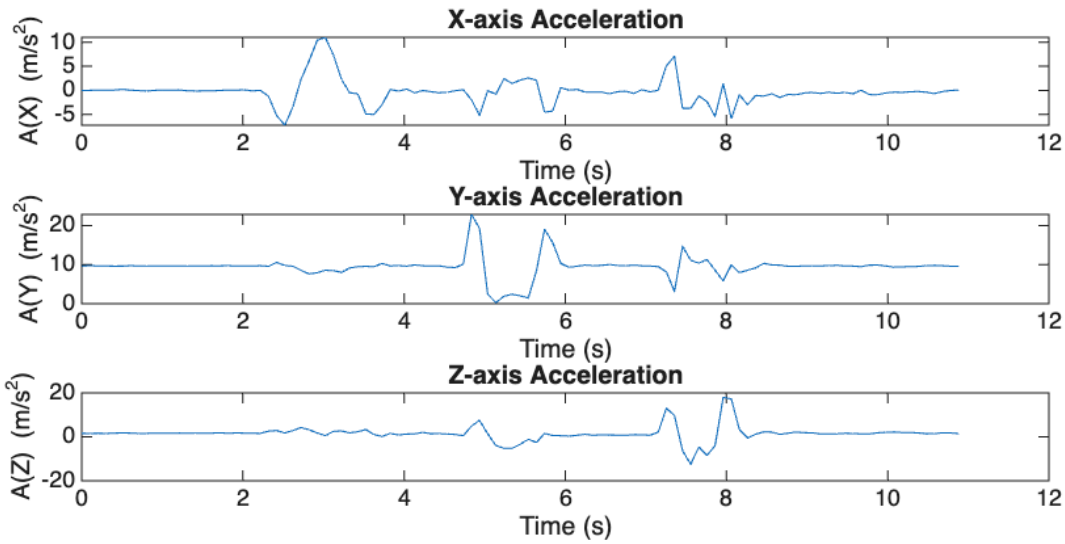

Figure 7: An example plot from an experiment to determine the X, Y, Z axes on a smartphone.

In your lab report, include the plot(s) of the data you collected in step 3.1 above. Include a hand-drawn sketch of your phone in your lab report with the axes labeled. Label both the positive and negative examples. An example, without axes labeled, is shown in Figure 8 below:

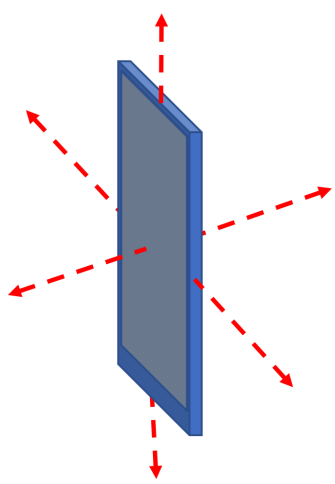

Figure 8: Template sketch for determining phone axes. Include a version of this in your lab report with axes clearly labeled.

Step 3.2: Validation

It's good practice to check that measured sensor values accord with your physical intuition about a problem. Recall that acceleration is the *rate of change* in velocity over time. That is, if

an object is traveling at a constant velocity its acceleration should be zero. Place your phone face up on a flat surface and collect a 10 second accelerometer log. Plot the X, Y, and Z acceleration values. What do you notice?

In your lab report include: the plot created above and a short (1 paragraph max) reflection on the results. Your reflection should address whether the results accord with your physical intuition about acceleration.

What to Submit

- Your script logging.m. Your script must be called “logging.m” and it must be submitted as a .m file.
- A lab report containing:
    - Plots of the experiment from Step 3.1 used to determine your phone’s X,Y, and Z axes. Circle or otherwise highlight (by hand is fine) the motif in the plot you used to identify the axes of your phone.
    - A hand-drawn sketch, similar to figure 8, showing the X, Y, and Z axes on your phone.
    - Plots of the experiment from Step 3.2
    - A 1 paragraph reflection on the data collected in Step 3.2.

Note: it is a violation of the academic integrity policy to submit someone else's plots as your own. We will run your code. If you submit plots, but your code does not run, we will be suspicious.

Rubric

- **Code**: `logging.m` runs without errors, correctly collects data and produces plots.
- **Plots and Sketch**:
    - Plots are clear and annotated with motifs used to identify phone axes.
    - Hand-drawn sketch with axes correctly labeled.
- **Reflection**:
    - Reflection meaningfully discusses whether results match intuition. If not, provide reasonable speculation as to why.